

Data Structure & Algorithms

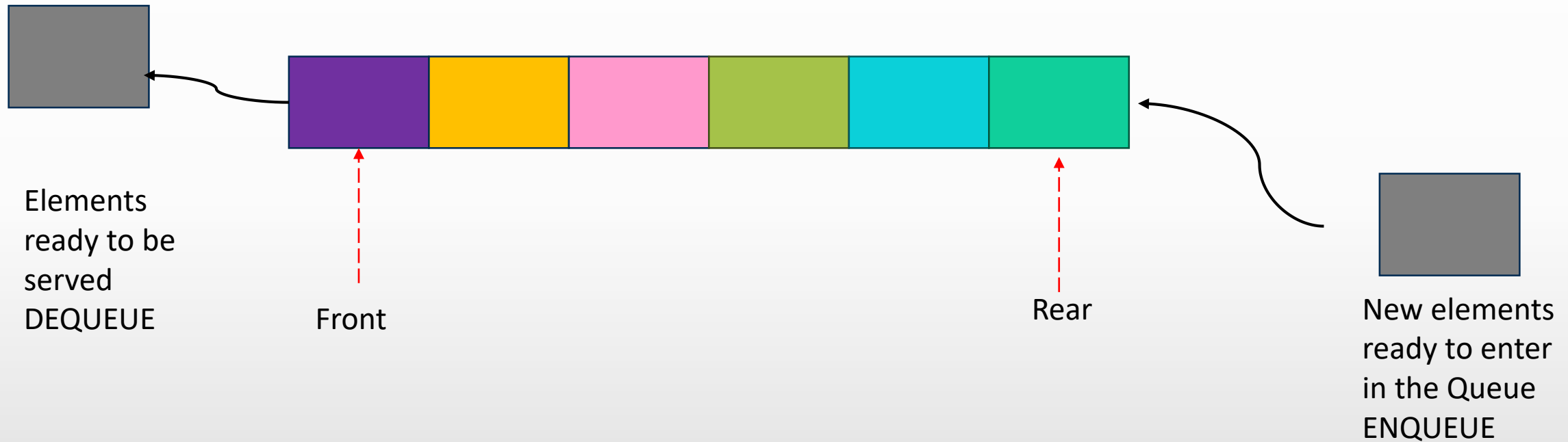
Queue

Introduction

- A queue is a data structure used for storing data (similar to Linked Lists and Stacks).
- In general, a queue is a line of people or things waiting to be served in sequential order starting at the beginning of the line or sequence.

Definition: A queue is an ordered list in which insertions are done at one end (rear) and deletions are done at other end (front). The first element to be inserted is the first one to be deleted. Hence, it is called First in First out (FIFO) or Last in Last out (LIFO) list.

When an element is inserted in a queue, the concept is called EnQueue, and when an element is removed from the queue, the concept is called DeQueue.



Characteristics of Queue

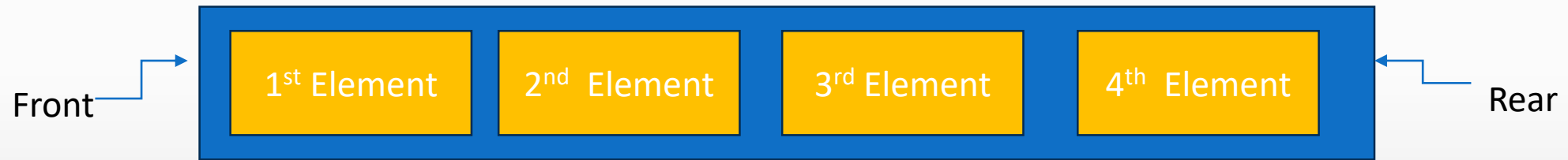
- Queues are widely used as waiting lists for a single shared resource like printer, disk, CPU.
- Queues are used in asynchronous transfer of data (where data is not being transferred at the same rate between two processes) for eg. pipes, file IO, sockets.
- Queues are used as buffers in most of the applications like MP3 media player, CD player, etc.
- Queue are used to maintain the play list in media players in order to add and remove the songs from the play-list.
- Queues are used in operating systems for handling interrupts.

Types of Queue

- Simple Queue or Linear Queue
- Circular Queue
- Priority Queue
- Double Ended Queue (or Deque)

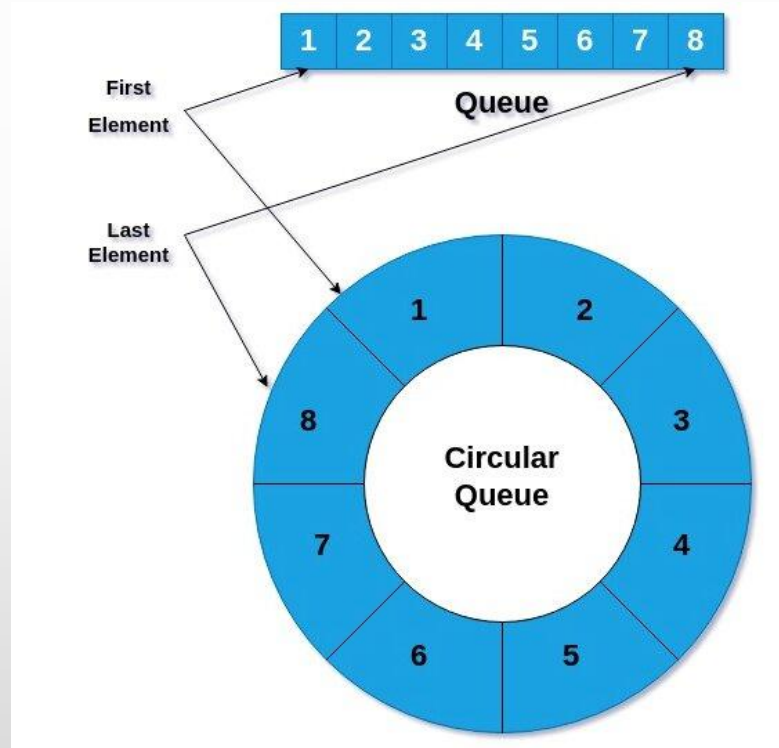
Linear or Simple Queue

In Linear Queue, an insertion takes place from one end while the deletion occurs from another end. The end at which the insertion takes place is known as the rear end, and the end at which the deletion takes place is known as front end. It strictly follows the FIFO rule.



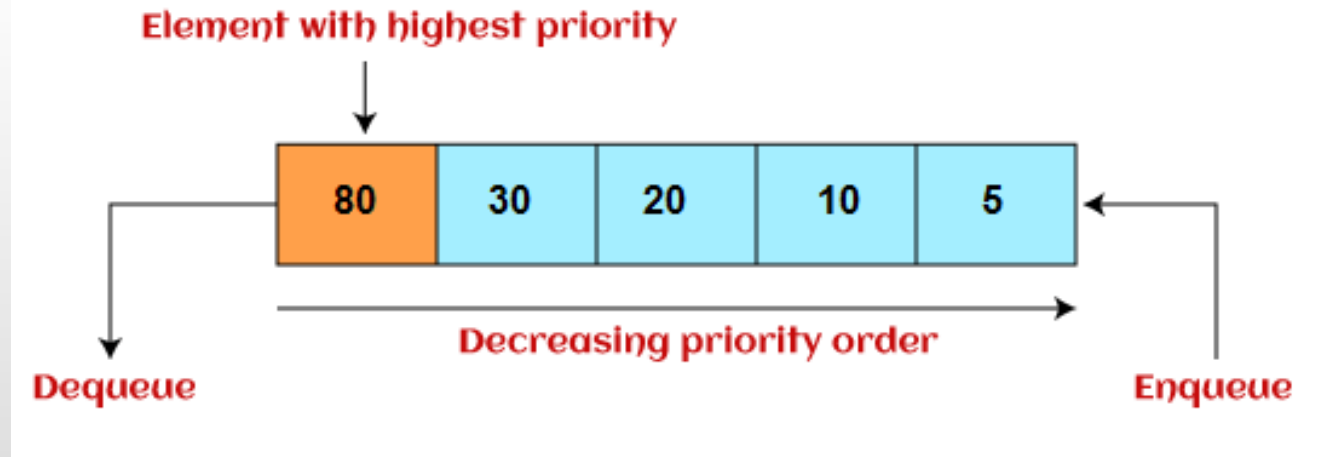
Circular Queue

In Circular Queue, all the nodes are represented as circular. It is similar to the linear Queue except that the last element of the queue is connected to the first element. It is also known as Ring Buffer, as all the ends are connected to another end.



Priority Queue

- It is a special type of queue in which the elements are arranged based on the priority.
- Suppose some elements occur with the same priority, they will be arranged according to the FIFO principle.

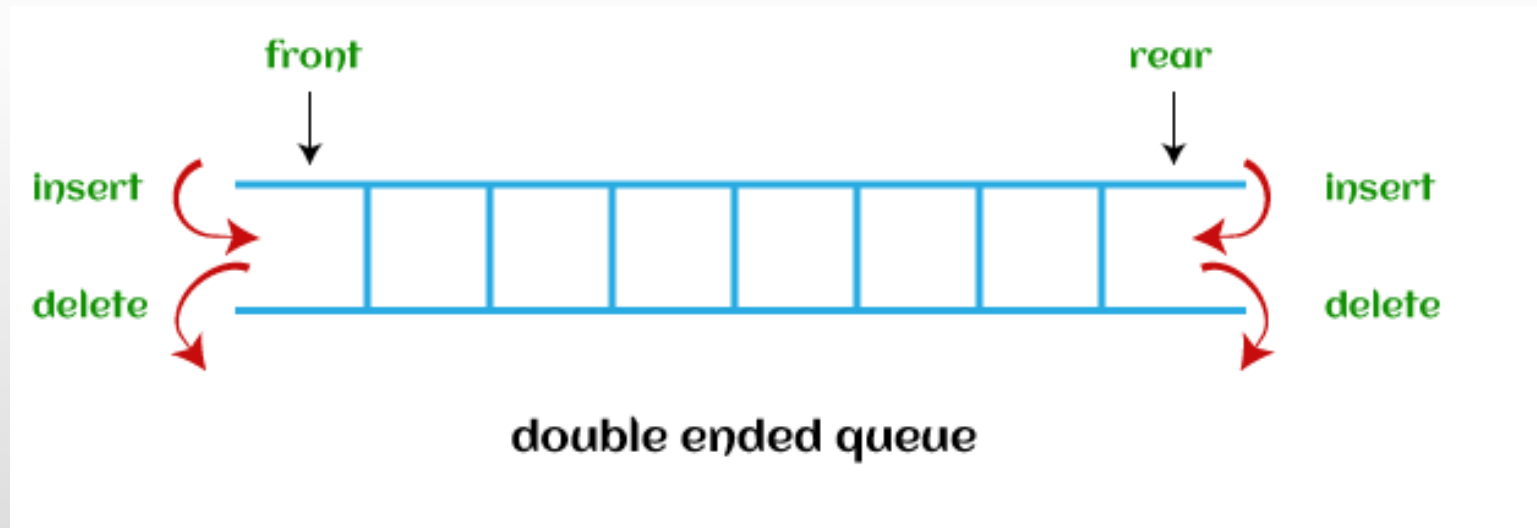


There are two types of priority queue

- **Ascending priority queue** - In ascending priority queue, elements can be inserted in arbitrary order, but only smallest can be deleted first. Suppose an array with elements 7, 5, and 3 in the same order, so, insertion can be done with the same sequence, but the order of deleting the elements is 3, 5, 7.
- **Descending priority queue** - In descending priority queue, elements can be inserted in arbitrary order, but only the largest element can be deleted first. Suppose an array with elements 7, 3, and 5 in the same order, so, insertion can be done with the same sequence, but the order of deleting the elements is 7, 5, 3.

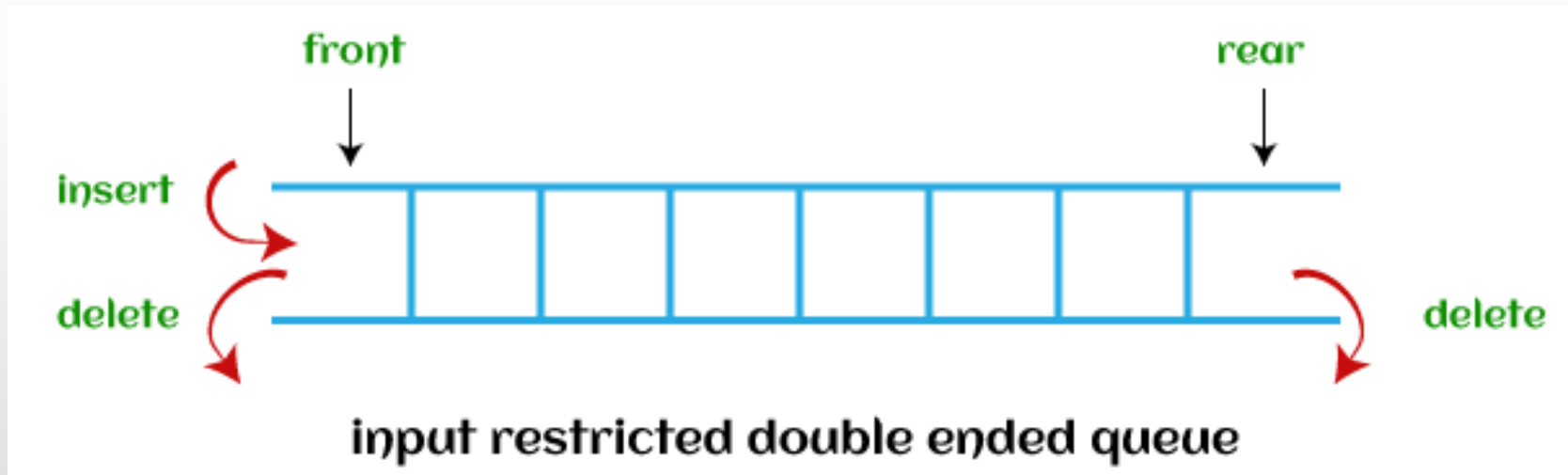
DEQUEUE

- In Deque or Double Ended Queue, insertion and deletion can be done from both ends of the queue either from the front or rear.
- We can insert and delete elements from both front and rear ends of the queue.

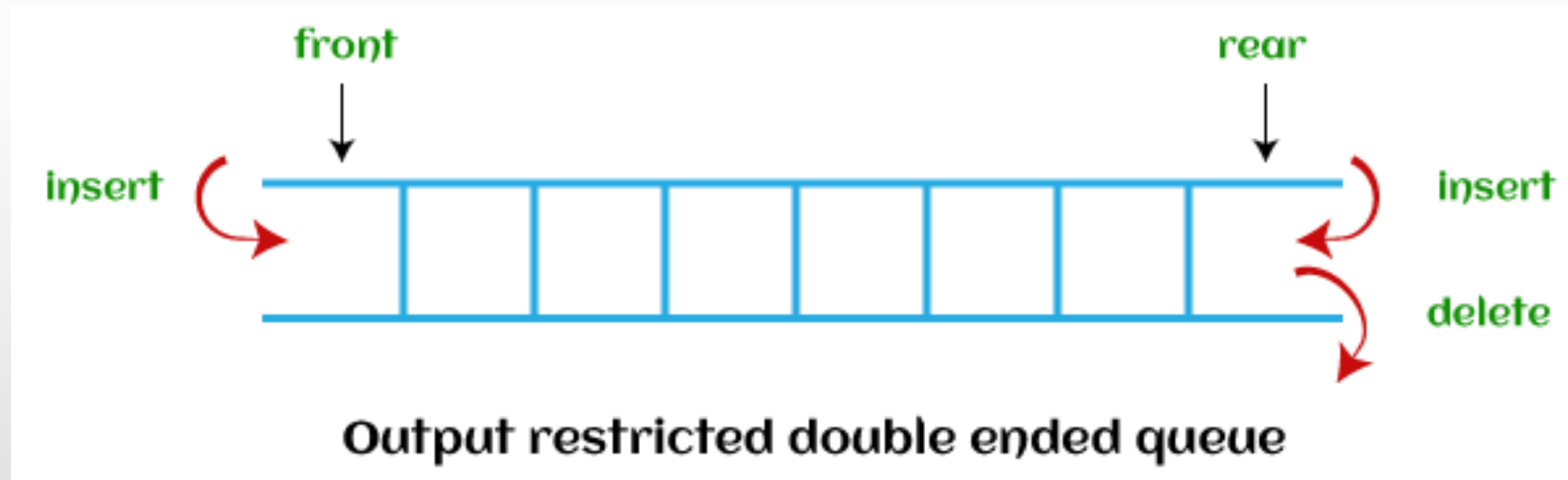


There are two types of deque that are discussed as follows –

- Input restricted deque - As the name implies, in input restricted queue, insertion operation can be performed at only one end, while deletion can be performed from both ends.



Output restricted deque - As the name implies, in output restricted queue, deletion operation can be performed at only one end, while insertion can be performed from both ends.



Operations on Queue

Enqueue

Dequeue

Peek

Queue overflow (isfull)

Queue underflow (isempty)

Algorithm to insert any element in a queue

Check if the queue is already full by comparing rear to max - 1. if so, then return an overflow error.

If the item is to be inserted as the first element in the list, in that case set the value of front and rear to 0 and insert the element at the rear end.

Otherwise keep increasing the value of rear and insert each element one by one having rear as the index.

Algorithm

Step 1: IF REAR = MAX - 1

Write OVERFLOW

Go to step

[END OF IF]

Step 2: IF FRONT = -1 and REAR = -1

SET FRONT = REAR = 0

ELSE

SET REAR = REAR + 1

[END OF IF]

Step 3: Set QUEUE[REAR] = NUM

Step 4: EXIT

```
void insert (int queue[], int max, int front, int rear, int item)
{
    if (rear + 1 == max)
    {
        printf("overflow");
    }
    else
    {
        if(front == -1 && rear == -1)
        {
            front = 0;
            rear = 0;
        }
        else
        {
            rear = rear + 1;
        }
        queue[rear]=item;
    }
}
```

Algorithm to delete an element from the queue

If, the value of front is -1 or value of front is greater than rear , write an underflow message and exit.

Otherwise, keep increasing the value of front and return the item stored at the front end of the queue at each time

Algorithm

Step 1: IF FRONT = -1 or FRONT > REAR

Write UNDERFLOW

ELSE

SET VAL = QUEUE[FRONT]

SET FRONT = FRONT + 1

[END OF IF]

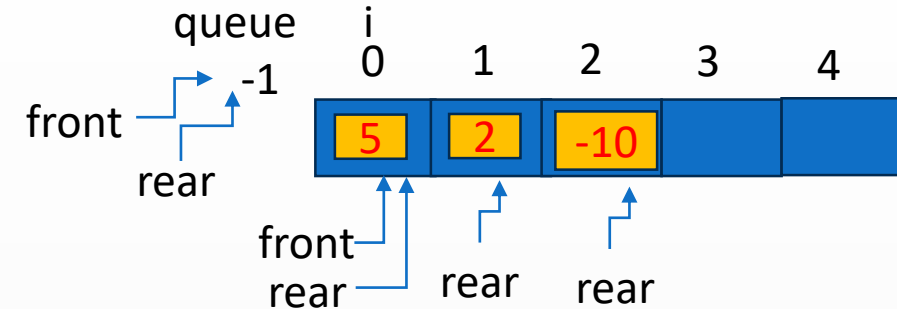
Step 2: EXIT

```
int delete (int queue[], int max, int front, int rear)
{
    int y;
    if (front == -1 || front > rear)
    {
        printf("underflow");
    }
    else
    {
        y = queue[front];
        if(front == rear)
        {
            front = rear = -1;
        }
        else
            front = front + 1;
    }
    return y;
}
```

Implementation of Queue using Arrays

```
void main()  
{  
    enqueue(5);  
    enqueue(2);  
    enqueue(-10);  
    display();  
    peek();  
    dequeue();  
    peek();  
    display();  
}
```

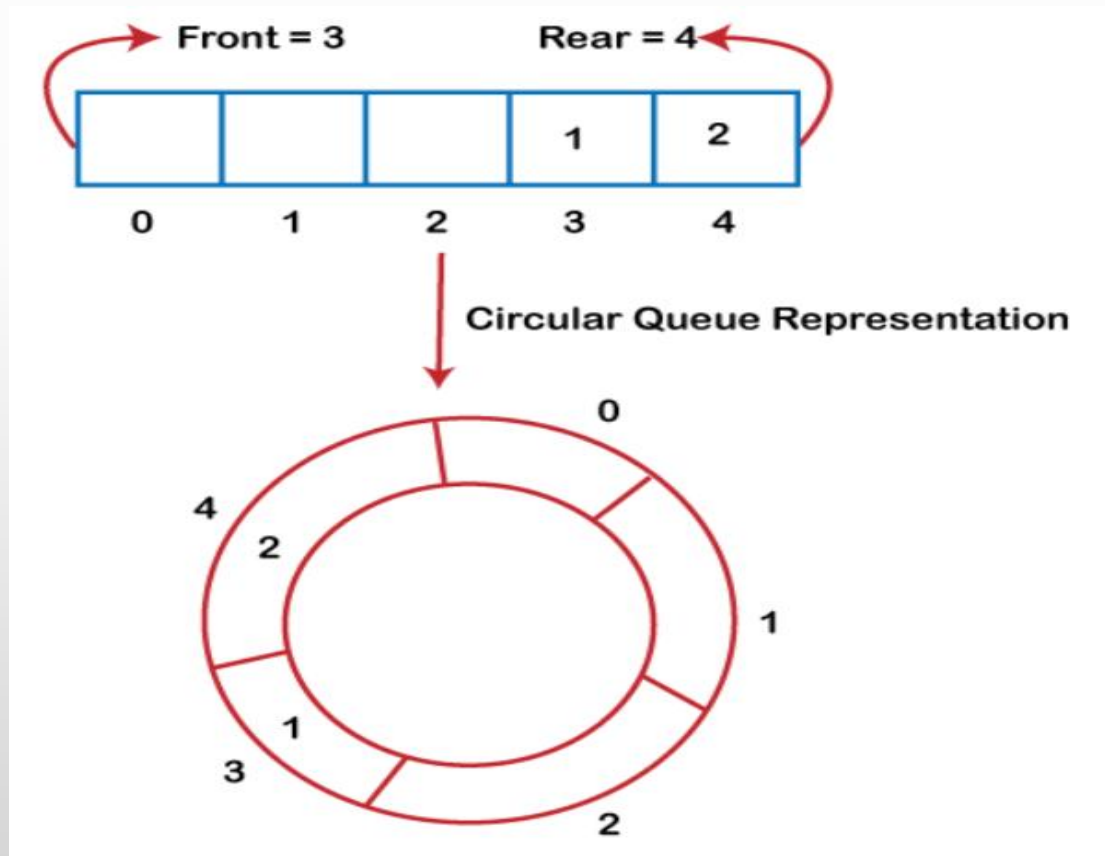
```
#define N 5  
int queue[N];  
int front=-1;  
int rear=-1;  
void enqueue(int x)  
{  
    if (rear==N-1)  
    {  
        printf("overflow");  
    }  
    else if(front ==-1 && rear==-1)  
    {  
        front=rear=0;  
        queue[rear]=x;  
    }  
    else{  
        rear++;  
        queue[rear]=x;  
    }  
}
```



```
void display()  
{  
    int i;  
    if (front=rear=-1){  
        printf("queue is empty");  
    }  
    else{  
        for(i=front;i<rear+1;i++)  
        {  
            printf("%d",queue[i]);  
        }  
    }  
}
```


Circular Queue

There was one limitation in the array implementation of Queue. If the rear reaches to the end position of the Queue then there might be possibility that some vacant spaces are left in the beginning which cannot be utilized. So, to overcome such limitations, the concept of the circular queue was introduced.



A circular queue is similar to a linear queue as it is also based on the FIFO (First In First Out) principle except that the last position is connected to the first position in a circular queue that forms a circle. It is also known as a **Ring Buffer**.

Operations on Circular Queue

The following are the operations that can be performed on a circular queue:

- **Front:** It is used to get the front element from the Queue.
- **Rear:** It is used to get the rear element from the Queue.
- **enQueue(value):** This function is used to insert the new value in the Queue. The new element is always inserted from the rear end.
- **deQueue():** This function deletes an element from the Queue. The deletion in a Queue always takes place from the front end.

Operations on Circular Queue

Enqueue operation

The steps of enqueue operation are given below:

- First, we will check whether the Queue is full or not.
- Initially the front and rear are set to -1. When we insert the first element in a Queue, front and rear both are set to 0.
- When we insert a new element, the rear gets incremented, i.e., ***rear=rear+1***.

EnQueue Operation

Algorithm to insert an element in a circular queue

- **Step 1:** IF $(\text{REAR} + 1) \% \text{MAX} = \text{FRONT}$
Write " OVERFLOW "
Goto step 4
[End OF IF]
- **Step 2:** IF $\text{FRONT} = -1$ and $\text{REAR} = -1$
SET $\text{FRONT} = \text{REAR} = 0$
ELSE IF $\text{REAR} = \text{MAX} - 1$ and $\text{FRONT} \neq 0$
SET $\text{REAR} = 0$
ELSE
SET $\text{REAR} = (\text{REAR} + 1) \% \text{MAX}$
[END OF IF]
- **Step 3:** SET $\text{QUEUE}[\text{REAR}] = \text{VAL}$
- **Step 4:** EXIT

Deque Operation

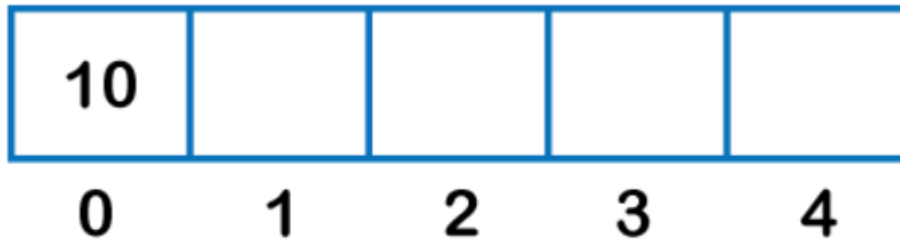
- The steps of dequeue operation are given below:
- First, we check whether the Queue is empty or not. If the queue is empty, we cannot perform the dequeue operation.
- When the element is deleted, the value of front gets decremented by 1.
- If there is only one element left which is to be deleted, then the front and rear are reset to -1.

Algorithm to delete an element from the circular queue

- **Step 1:** IF FRONT = -1
Write " UNDERFLOW "
Goto Step 4
[END of IF]
- **Step 2:** SET VAL = QUEUE[FRONT]
- **Step 3:** IF FRONT = REAR
SET FRONT = REAR = -1
ELSE
IF FRONT = MAX -1
SET FRONT = 0
ELSE
SET FRONT = FRONT + 1
[END of IF]
[END OF IF]
- **Step 4:** EXIT

Front = -1

Rear = -1



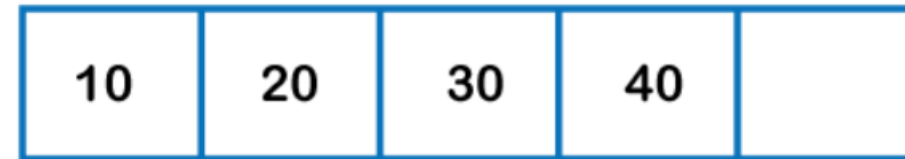
Front = 0

Rear = 0



Front = 0

Rear = 2



Front = 0

Rear = 3



0

1

2

3

4



Front = 0



Rear = 4



0

1

2

3

4

dequeue



Front = 2



Rear = 4



0

1

2

3

4



Rear



Front



0

1

2

3

4

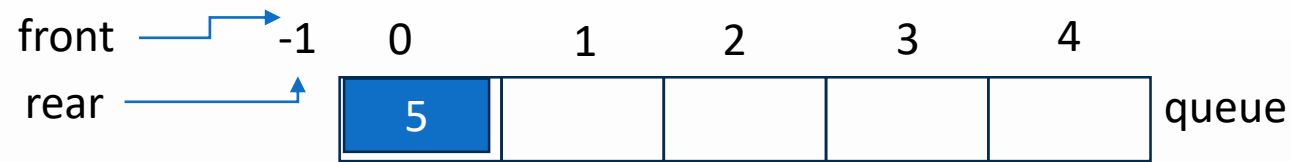


Rear



Front

Circular Queue(Enqueue)

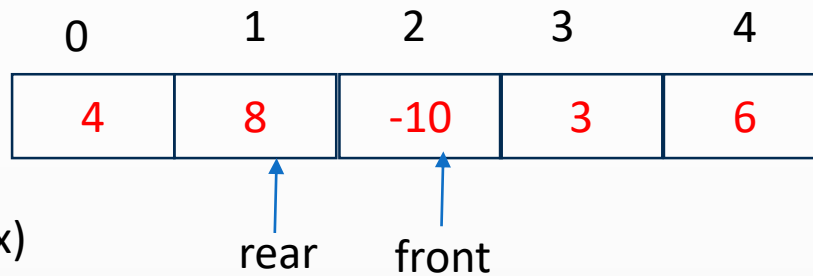


```
#define N 5  
int queue[N];  
int front=-1;  
int rear=-1;
```

```
void enqueue(int x)  
{  
    if (front == -1 && rear == -1)  
    {  
        front = rear = 0;  
    }
```

```
    else if ((rear + 1) % MAX_SIZE == front)  
    {  
        printf("queue is full\n");  
        return;  
    }
```

```
    else  
        rear = (rear + 1) % MAX_SIZE;  
    queue[rear] = x;  
}
```



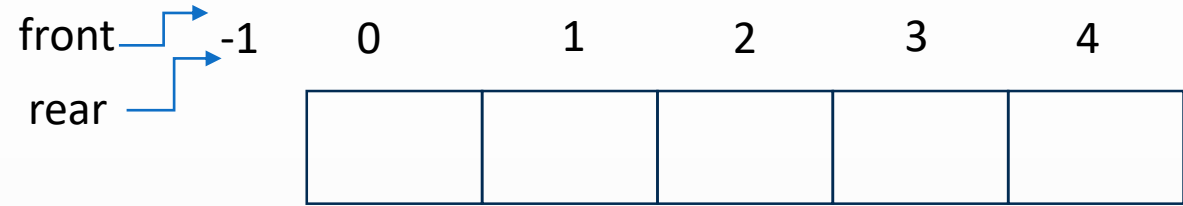
```
void main()  
{  
    enqueue(5);  
    enqueue(2);  
    enqueue(-10);  
    enqueue(3);  
    display();  
    dequeue();  
}
```


Circular Queue(Dequeue)

```
void dequeue()
{
    if (front == -1 && rear == -1)
    {
        printf("queue is empty \n");
        return;
    }

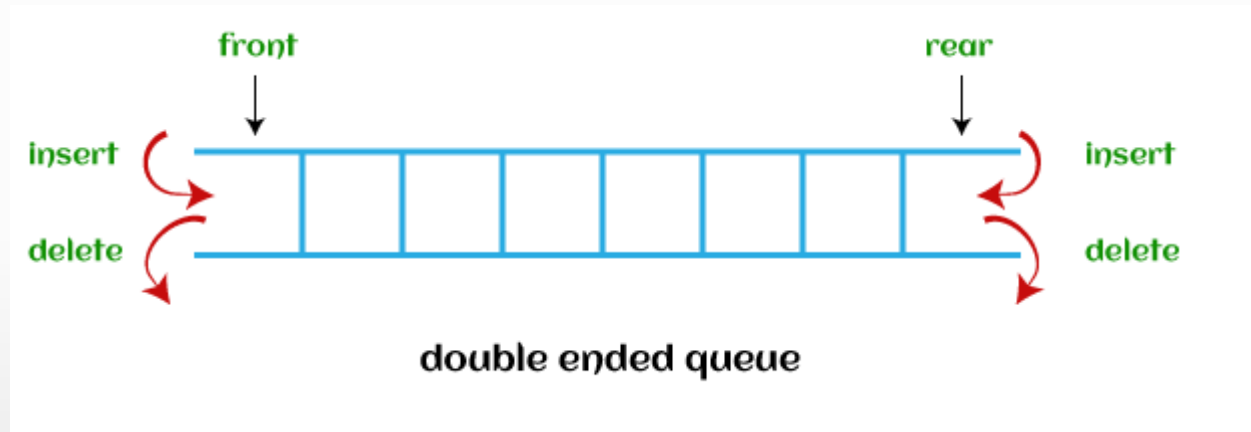
    else if (front == rear)
    {
        front = rear = -1;
    }

    else
    {
        front = (front + 1) % MAX_SIZE;
    }
}
```



Double Ended Queue

Deque or Double Ended Queue is a type of queue in which insertion and removal of elements can either be performed from the front or the rear.



Deque can be implemented using circular array. In a circular array, if the array is full, we start from the beginning.

Mainly the following four basic operations are performed on queue:

- insertFront():** Adds an item at the front of Deque.
- insertRear():** Adds an item at the rear of Deque.
- deleteFront():** Deletes an item from front of Deque.
- deleteRear():** Deletes an item from rear of Deque.

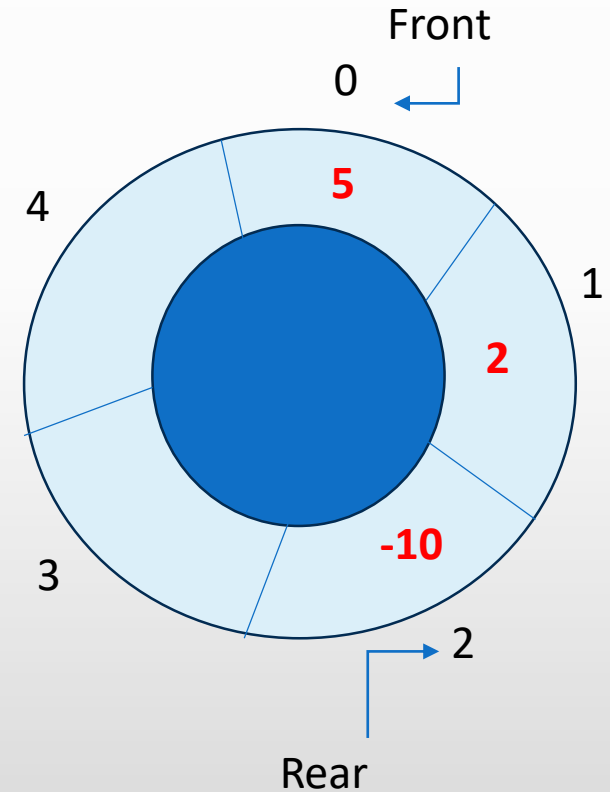
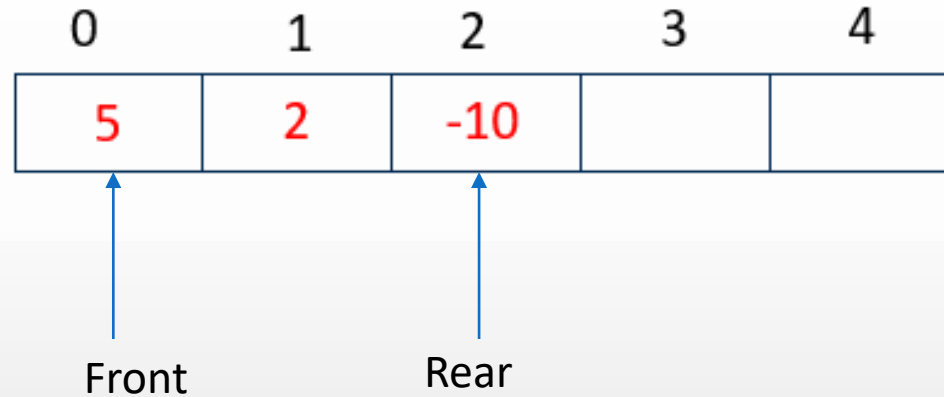
Applications of deque

- Deque can be used as both stack and queue, as it supports both operations.
- Deque can be used as a palindrome checker means that if we read the string from both ends, the string would be the same.

InsertFront()

This operation adds an element at the front.

1. Check the position of the front

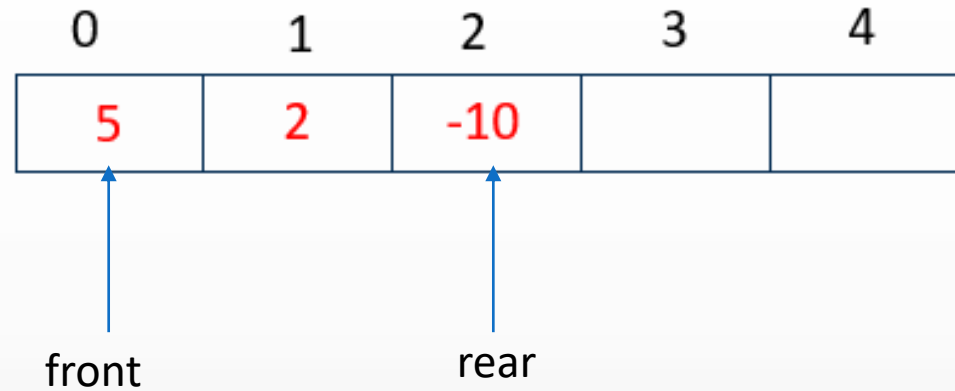


- a) if $\text{front} < 1$, reinitialize $\text{front} = n-1$ (last index)
- b) Else, $\text{front}--$.
- c) Add $\text{queue}[\text{front}] = x$;

InsertRear()

This operation adds an element to the rear.

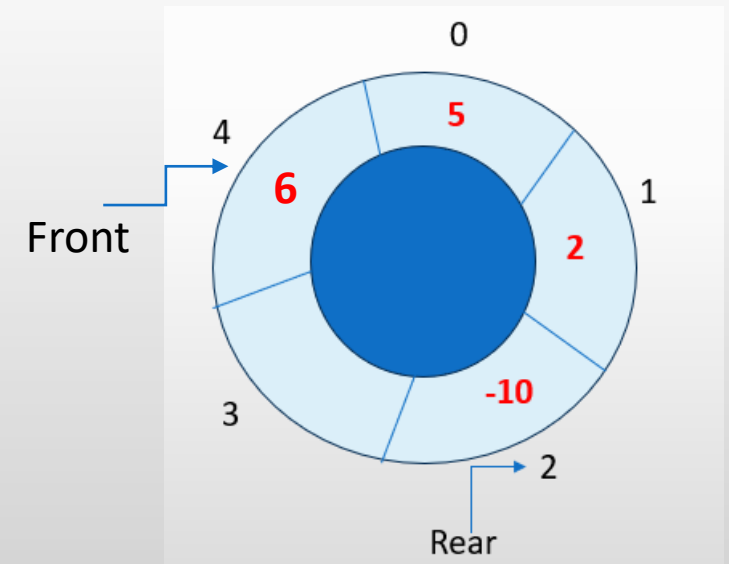
1. First we check deque if Full or Not
Rear == N-1 , reinitialize rear=0;
2. rear++;
3. Queue[rear]=x;



DeleteFront()

The operation deletes an element from the front.

1. Check if the deque is empty.
2. If the deque has only one element (i.e. $\text{front} = \text{rear}$), set $\text{front} = -1$ and $\text{rear} = -1$.
3. Else if front is at the end (i.e. $\text{front} = n - 1$), set go to the front $\text{front} = 0$.
4. Else, $\text{front} = \text{front} + 1$



DeleteRear()

This operation deletes an element from the rear.

1. Check if the deque is empty
2. If the deque has only one element (i.e. $\text{front} = \text{rear}$), set $\text{front} = -1$ and $\text{rear} = -1$.
3. If rear is at the front (i.e. $\text{rear} = 0$), set go to the front $\text{rear} = n - 1$.
4. Else, $\text{rear}--$;



Thank You